Listing 5.3 Data for blind auction (BlindAuction.sol)

```
pragma solidity >=0.4.22 <=0.6.0;

contract BlindAuction {

    struct Bid {                                    ❶
        bytes32 blindedBid;
        uint deposit;
    }

    // state will be set by beneficiary             ❷
    enum Phase {Init, Bidding, Reveal, Done}
    Phase public state = Phase.Init;

    address payable beneficiary; // owner           ❸
    mapping(address => Bid) bids;                   ❹


    address public highestBidder;                   ❺
    uint public highestBid = 0;                     ❺

    mapping(address => uint) depositReturns;        ❻
```

❶ Bid details
❷ Auction state details
❸ Contract deployer is the beneficiary
❹ Only one bid per address
❺ Details of the highest bidder
❻ Returns of deposits for nonwinners

The blind auction problem has two main rules: the beneficiary decides on the timing of the starting, ending, `Bidding`, and `Reveal` phases of the auction, and only the beneficiary can change the phase from one to the next. These conditions are shown in the following listing, implemented as the modifiers `validPhase` and `onlyBeneficiary`. You can insert them into code (listing 5.3) already loaded in the Remix IDE.

Listing 5.4 Modifiers for blind auction (BlindAuction.sol)

```
   // modifiers
   modifier validPhase(Phase reqPhase)          ❶
    { require(state == reqPhase);
      _;
    }

   modifier onlyBeneficiary()
   { require(msg.sender == beneficiary);        ❷
```